

Assignment 6

Hadeer Ghoneim

Verification Diploma

Design

```
8  import shared_pkg::*;
9  module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
10 input clk, rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
11 input [2:0] opcode;
12 input signed [2:0] A, B;
13 output reg [15:0] leds;
14 output reg [5:0] out;
15
16 reg cin_reg, red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
17 reg [2:0] opcode_reg, A_reg, B_reg;
18
19 wire invalid_red_op, invalid_opcode, invalid;
20 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
21 assign invalid_opcode = opcode_reg[1] & opcode_reg[2]; // Fix
22 assign invalid = invalid_red_op | invalid_opcode;
23
24 always @(posedge clk or posedge rst) begin
25     if(rst) begin
26         leds <= 0;
27         out <= 0;
28         cin_reg <= 0;
29         red_op_B_reg <= 0;
30         red_op_A_reg <= 0;
31         bypass_B_reg <= 0;
32         bypass_A_reg <= 0;
33         direction_reg <= 0;
34         serial_in_reg <= 0;
35         opcode_reg <= 0;
36         A_reg <= 0;
37         B_reg <= 0;
38     end
end
```

```

39 end
40 else begin
41     if (invalid && !(bypass_A_reg || bypass_B_reg)) // FIX
42         leds <= ~leds;
43     else
44         leds <= 0;
45     cin_reg <= cin;
46     red_op_B_reg <= red_op_B;
47     red_op_A_reg <= red_op_A;
48     bypass_B_reg <= bypass_B;
49     bypass_A_reg <= bypass_A;
50     direction_reg <= direction;
51     serial_in_reg <= serial_in;
52     opcode_reg <= opcode;
53     A_reg <= A;
54     B_reg <= B;
55
56     if (bypass_A_reg && bypass_B_reg)
57         out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
58     else if (bypass_A_reg)
59         out <= A_reg;
60     else if (bypass_B_reg)
61         out <= B_reg;
62     else if (invalid) // FIX
63         out <= 0; // FIX
64     else begin
65         case (opcode_reg) // FIX
66             3'h0: begin
67                 if (red_op_A_reg && red_op_B_reg)
68                     out = (INPUT_PRIORITY == "A")? |A_reg: |B_reg; // FIX
69                 else if (red_op_A_reg)
70                     out <= |A_reg; // FIX
71                 else if (red_op_B_reg)
72                     out <= |B_reg; // FIX
73                 else
74                     out <= A reg | B reg; // FIX

```

```

75     end
76     3'h1: begin
77         if (red_op_A_reg && red_op_B_reg)
78             out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg; // Fix
79         else if (red_op_A_reg)
80             out <= ^A_reg; // Fix
81         else if (red_op_B_reg)
82             out <= ^B_reg; // Fix
83         else
84             out <= A_reg ^ B_reg; // Fix
85     end
86     3'h2: begin
87         if (FULL_ADDER == "ON")
88             out <= A_reg + B_reg + cin_reg;
89         else
90             out <= A_reg + B_reg;
91     end
92     3'h3: out <= A_reg * B_reg;
93     3'h4: begin
94         if (direction_reg)
95             out <= {out[4:0], serial_in_reg};
96         else
97             out <= {serial_in_reg, out[5:1]};
98     end
99     3'h5: begin
100         if (direction_reg)
101             out <= {out[4:0], out[5]};
102         else
103             out <= {out[0], out[5:1]};
104     end
105 endcase
106 end
107 end
108 end
109
110 endmodule

```

Golden Model

```

1  import shared_pkg::*;
2
3  module ALSU_ref (A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
4  input clk, rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5  //input opcode_e opcode;
6  input [2:0] opcode;
7  input signed [2:0] A, B;
8  output reg [15:0] leds;
9  output reg [5:0] out;
10
11  reg cin_reg, red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
12  reg [2:0] A_reg, B_reg;
13  reg [2:0] opcode_reg;
14
15  wire invalid, invalid_c1, invalid_c2;
16  assign invalid_c1 = (opcode_reg == 6 || opcode_reg == 7);
17  assign invalid_c2 = (red_op_A_reg || red_op_B_reg) && (opcode_reg != 0 && opcode_reg != 1);
18  assign invalid = invalid_c1 || invalid_c2;
19
20  always @(posedge clk or posedge rst) begin
21      if (rst) begin
22          out <= 0;
23          leds <= 0;
24          cin_reg <= 0;
25          red_op_A_reg <= 0;
26          red_op_B_reg <= 0;
27          bypass_A_reg <= 0;
28          bypass_B_reg <= 0;
29          direction_reg <= 0;
30          serial_in_reg <= 0;
31          opcode_reg <= 0;
32          A_reg <= 0;
33          B_reg <= 0;
34      end

```

```

35     else begin
36         if (invalid&&!(bypass_A_reg || bypass_B_reg)) leds <= ~leds;
37         else leds <= 0;
38
39         cin_reg <= cin;
40         red_op_A_reg <= red_op_A;
41         red_op_B_reg <= red_op_B;
42         bypass_A_reg <= bypass_A;
43         bypass_B_reg <= bypass_B;
44         direction_reg <= direction;
45         serial_in_reg <= serial_in;
46         opcode_reg <= opcode;
47         A_reg <= A;
48         B_reg <= B;
49
50         if (bypass_A_reg && bypass_B_reg)
51             out <= (INPUT_PRIORITY=="A")? A_reg:B_reg;
52         else if (bypass_A_reg)
53             out <= A_reg;
54         else if (bypass_B_reg)
55             out <= B_reg;
56         else if (invalid)
57             out <= 0;
58         else if (opcode_reg == 0) begin
59             if (red_op_A_reg && red_op_B_reg)
60                 out <= (INPUT_PRIORITY=="A")? (|A_reg):(|B_reg);
61             else if (red_op_A_reg)
62                 out <= |A_reg;
63             else if (red_op_B_reg)
64                 out <= |B_reg;
65             else
66                 out <= A_reg|B_reg;
67         end

```

```

67     end
68     else if (opcode_reg == 1) begin
69         if (red_op_A_reg && red_op_B_reg)
70             out <= (INPUT_PRIORITY=="A")? (^A_reg):(^B_reg);
71         else
72             out <= (red_op_A_reg)? (^A_reg): (red_op_B_reg)? (^B_reg):(A_reg^B_reg);
73         end
74     else if (opcode_reg == 2)
75         out <= (FULL_ADDER=="ON")? (A_reg + B_reg + cin_reg):(A_reg + B_reg);
76     else if (opcode_reg == 3)
77         out <= A_reg * B_reg;
78     else if (opcode_reg == 4) begin
79         if (direction_reg)
80             out <= {out[4:0], serial_in_reg};
81         else
82             out <= {serial_in_reg, out[5:1]};
83         end
84     else if (opcode_reg == 5) begin
85         if (direction_reg)
86             out <= {out[4:0],out[5]};
87         else
88             out <= {out[0],out[5:1]};
89         end
90     end
91 end
92 endmodule
93

```

Interface

```

2  import shared_pkg::*;
3  interface ALSU_interface(input bit clk);
4  logic rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5  opcode_e opcode;
6  logic signed [2:0] A, B;
7  logic [15:0] leds;
8  logic [5:0] out;
9  logic [5:0] out_ref;
10 logic [15:0] leds_ref;
11 endinterface //ALSU_interface

```

shared_pkg

```
2  package shared_pkg;
3  import uvm_pkg::*;
4  `include "uvm_macros.svh"
5  parameter INPUT_PRIORITY = "A";
6  parameter FULL_ADDER = "ON";
7
8  typedef enum reg [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;
9  parameter MAXPOS = 3;
10 parameter ZERO = 0;
11 parameter MAXNEG = -4;
12
13 parameter HOBBIT_LOOP = 1000;
14 parameter DWARF_LOOP = 10_000;
15 parameter HUMAN_LOOP = 20_000;
16 parameter GIANT_LOOP = 30_000;
17
18 bit alwaysValid = 0;
19 int RESET_DIST = 5;
20 endpackage
```

Configuration

```
2  package config_pkg;
3  import shared_pkg::*;
4  import uvm_pkg::*;
5  `include "uvm_macros.svh"
6
7  // Configuration class
8  class ALSU_config extends uvm_object;
9      `uvm_object_utils(ALSU_config)
10     virtual ALSU_interface v_if;
11     function new(string name = "ALSU_config");
12         super.new(name);
13     endfunction //new()
14 endclass //ALSU_config
15 endpackage
```

sequence Item

```

1 package sequenceItem_pkg;
2 import shared_pkg::*;
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5 `define create_obj(type, name) type::type_id::create(name, this);
6
7 // Sequence Item class Valid and Invalid
8 class ALSU_sequenceItem extends uvm_sequence_item;
9     `uvm_object_utils(ALSU_sequenceItem)
10     logic clk;
11     rand logic rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
12     rand opcode_e opcode;
13     rand logic signed [2:0] A, B;
14     logic [15:0] leds;
15     logic [5:0] out;
16     logic [15:0] leds_ref;
17     logic [5:0] out_ref;
18
19     function new(string name = "ALSU_sequenceItem");
20         super.new(name);
21     endfunction //new()
22
23     // built in function
24     function string convert2string();
25         return $sformatf("%s, rst = %0d, A = %0d, B = %0d cin = %0d, red_op_A = %0d, red_op_B = %0d, bypass_A = %0d, bypass_B = %0d,
26             direction = %0d, serial_in = %0d \n out = %0d, out_ref = %0d",
27             super.convert2string(), rst, A, B, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, out, out_ref
28         );
29     endfunction
30     // my function to print ALSU inputs and outputs
31     function string convert2string_stim();
32         return $sformatf("rst = %0d, opcode = %0s, cin = %0d, red_op_A = %0d, red_op_B = %0d, bypass_A = %0d, bypass_B = %0d,
33             direction = %0d, serial_in = %0d \n out = %0d",
34             rst, opcode, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, out
35         );
36     endfunction
37

```

Activate Windows
Go to Settings to activate Windows.

```

38 ///////////////
39 // Constraint block //
40 ///////////////
41 constraint rules1_7 {
42     //rst constraint
43     rst dist {0:=100-RESET_DIST, 1:=RESET_DIST};
44
45     // Invalid cases constraint
46     opcode dist {INVALID_6:=5, INVALID_7:=5, [0:5]:90};
47
48     // A & B constraint when opcode is ADD or MULT
49     (opcode == MULT || opcode == ADD) -> A dist {MAXPOS:=20, ZERO:=10, MAXNEG:=20, [MAXNEG+1:MAXPOS-1]:50};
50     (opcode == MULT || opcode == ADD) -> B dist {MAXPOS:=20, ZERO:=10, MAXNEG:=20, [MAXNEG+1:MAXPOS-1]:50};
51
52     // A & B constraint when opcode is OR or XOR and red_op_A is high
53     ((opcode==XOR || opcode==OR) && red_op_A) -> A dist {3'b001:=30, 3'b10:=30, 3'b100:=30, [MAXNEG+1:MAXPOS-1]:10};
54     ((opcode==XOR || opcode==OR) && red_op_A) -> B == 0;
55
56     // A & B constraint when opcode is OR or XOR and red_op_B is high
57     ((opcode==XOR || opcode==OR) && red_op_B) -> A == 0;
58     ((opcode==XOR || opcode==OR) && red_op_B) -> B dist {3'b001:=30, 3'b10:=30, 3'b100:=30, [MAXNEG+1:MAXPOS-1]:10};
59
60     // Do not constraint the inputs A or B when the operation is shift or rotate
61     // it's achieved by default after the 2,3 and 4 Constraint achieved
62
63     // bypass constraint
64     bypass_A dist {0:=90, 1:=10};
65     bypass_B dist {0:=90, 1:=10};
66
67     // red_op constraint
68     red_op_A dist {0:=80, 1:=20};
69     red_op_B dist {0:=80, 1:=20};
70

```

Activate Windows


```

62
63     // bypass constraint
64     bypass_A dist {0:=90, 1:=10};
65     bypass_B dist {0:=90, 1:=10};
66
67     // red_op constraint
68     red_op_A dist {0:=80, 1:=20};
69     red_op_B dist {0:=80, 1:=20};
70
71     // A & B constraint when opcode is OR or XOR and red_op is low
72     ((opcode==XOR || opcode==OR) && ~red_op_A && ~red_op_B) -> A == ~B;
73 }
74
75 rand opcode_e arr [6];
76 constraint rules_8 {
77     foreach(arr[i])
78     |   arr[i] inside {OR, XOR, ADD, MULT, SHIFT, ROTATE};
79     unique {arr};
80     //foreach(arr[i])
81 }
82
83 constraint MAKE_bypass_red_rst_ZERO {
84     rst == 0;
85     bypass_A == 0; bypass_B == 0;
86     red_op_A == 0; red_op_B == 0;
87 }
88
89 constraint INPUT_PRIORITY_CONS {
90     red_op_A == red_op_B;
91     bypass_A == bypass_B;
92 }
93 endclass //ALSU_sequenceItem extends uvm_sequence_item
94 endpackage

```

sequence Item Valid

```

sequenceItem_valid.svh
1 package sequenceItem_valid_pkg;
2 import sequenceItem_pkg::*;
3 import shared_pkg::*;
4 import uvm_pkg::*;
5 `include "uvm_macros.svh"
6 `define create_obj(type, name) type::type_id::create(name, this);
7 // Sequence Item class Valid Cases Only
8 class ALSU_sequenceItem_valid extends ALSU_sequenceItem;
9     `uvm_object_utils(ALSU_sequenceItem_valid)
10
11     function new(string name = "ALSU_sequenceItem_valid");
12         super.new(name);
13     endfunction //new()
14
15     ////////// // Constraint block //
16     // Constraint block //
17     ////////// //
18     constraint rules1_7 {
19         //rst constraint
20         rst dist {0:=100-RESET_DIST, 1:=RESET_DIST};
21
22         // make opcode always valid
23         //opcode dist {INVALID_6:=5, INVALID_7:=5, [0:5]:/90};
24         opcode inside {OR, XOR, ADD, MULT, SHIFT, ROTATE};
25         // to avoid invalid cases
26         if (opcode==OR||opcode==XOR)
27             red_op_A dist {0:=90, 1:=10};
28         else
29             red_op_A == 0;
30         if (opcode==OR||opcode==XOR)
31             red_op_B dist {0:=90, 1:=10};
32         else
33             red_op_B == 0;
34

```

```

34
35 // A & B constraint when opcode is ADD or MULT
36 (opcode == MULT || opcode == ADD) -> A dist {MAXPOS:=20, ZERO:=10, MAXNEG:=20, [MAXNEG+1:MAXPOS-1]:/50};
37 (opcode == MULT || opcode == ADD) -> B dist {MAXPOS:=20, ZERO:=10, MAXNEG:=20, [MAXNEG+1:MAXPOS-1]:/50};
38
39 // A & B constraint when opcode is OR or XOR and red_op_A is high
40 ((opcode==XOR || opcode==OR) && red_op_A) -> A dist {3'b001:=30, 3'b010:=30, 3'b100:=30, [MAXNEG+1:MAXPOS-1]:/10};
41 ((opcode==XOR || opcode==OR) && red_op_A) -> B == 0;
42
43 // A & B constraint when opcode is OR or XOR and red_op_B is high
44 ((opcode==XOR || opcode==OR) && red_op_B) -> A == 0;
45 ((opcode==XOR || opcode==OR) && red_op_B) -> B dist {3'b001:=30, 3'b010:=30, 3'b100:=30, [MAXNEG+1:MAXPOS-1]:/10};
46
47 // Do not constraint the inputs A or B when the operation is shift or rotate
48 // it's achieved by default after the 2,3 and 4 Constraint achieved
49
50 // bypass constraint
51 bypass_A dist {0:=90, 1:=10};
52 bypass_B dist {0:=90, 1:=10};
53
54 // A & B constraint when opcode is OR or XOR and red_op is low
55 ((opcode==XOR || opcode==OR) && ~red_op_A && ~red_op_B) -> A == ~B;
56 }
57 endclass //ALSU_sequenceItem_valid extends uvm_sequence_item
58 endpackage

```

ALSU reset sequence

```

2  package rst_sequence_pkg;
3  import shared_pkg::*;
4  import sequenceItem_pkg::*;
5  import uvm_pkg::*;
6  `include "uvm_macros.svh"
7  `define create_obj(type, name) type::type_id::create(name);
8
9  class ALSU_reset_sequence extends uvm_sequence #(ALSU_sequenceItem);
10     `uvm_object_utils(ALSU_reset_sequence)
11     function new(string name = "ALSU_reset_sequence");
12         super.new(name);
13     endfunction //new()
14
15
16     ALSU_sequenceItem item;
17     // Main task
18     task body;
19         // Creat seq_item
20         item = `create_obj(ALSU_sequenceItem, "item")
21         item.A = 0; item.B = 0; item.opcode = OR;
22         item.cin = 0; item.serial_in = 0; item.direction = 0;
23         item.bypass_A = 0; item.bypass_B = 0;
24         item.red_op_A = 0; item.red_op_B = 0;
25         start_item(item);
26         item.rst = 1;
27         finish_item(item);
28     endtask
29 endclass //ALSU_reset_sequence extends uvm_sequence #(ALSU_sequenceItem)
30 endpackage

```

ALSU main sequence

```

2  package main_sequence_pkg;
3  import shared_pkg::*;
4  import uvm_pkg::*;
5  import sequenceItem_pkg::*;
6  `include "uvm_macros.svh"
7  `define create_obj(type, name) type::type_id::create(name);
8
9  class ALSU_main_sequence extends uvm_sequence #(ALSU_sequenceItem);
10     `uvm_object_utils(ALSU_main_sequence)
11     function new(string name = "ALSU_main_sequence");
12         super.new(name);
13     endfunction //new()
14
15     ALSU_sequenceItem item;
16     // Main task
17     task body();
18         repeat(GIANT_LOOP) begin
19             item = `create_obj(ALSU_sequenceItem, "item") // Creat seq_item
20             // edit constraint mode
21             item.constraint_mode(0);
22             item.rules1_7.constraint_mode(1);
23
24             start_item(item);
25             assert (item.randomize());
26             finish_item(item);
27         end
28
29         repeat(HOBBIT_LOOP) begin
30             item = `create_obj(ALSU_sequenceItem, "item") // Creat seq_item
31             // edit constraint mode
32             item.constraint_mode(0);
33             item.INPUT_PRIORITY_CONS.constraint_mode(1);
34
35             start_item(item);
36             assert (item.randomize());
37             finish_item(item);
38         end

```

```

39
40     // make opcode onley valid
41     alwaysValid = 1;
42     repeat(DWARF_LOOP) begin
43         item = `create_obj(ALSU_sequenceItem, "item") // Creat seq_item
44         // edit constraint mode
45         item.constraint_mode(0);
46         item.rules_8.constraint_mode(1);
47         item.MAKE_bypass_red_rst_ZERO.constraint_mode(1);
48         // randomization
49         assert (item.randomize());
50         // loop for opcode
51         foreach(item.arr[i]) begin
52             start_item(item);
53             item.opcode = item.arr[i];
54             finish_item(item);
55         end
56     end
57 endtask
58 endclass //ALSU_main_sequence extends uvm_sequence #(ALSU_sequenceItem)
59 endpackage

```

Sequencer

```

1  package sequencer_pkg;
2  import uvm_pkg::*;
3  import sequenceItem_pkg::*;
4  `include "uvm_macros.svh"
5  `define create_obj(type, name) type::type_id::create(name, this);
6
7  class sequencer extends uvm_sequencer #(ALSU_sequenceItem);
8      `uvm_component_utils(sequencer)
9      function new(string name = "sequencer", uvm_component parent = null);
10         super.new(name, parent);
11     endfunction
12 endclass
13 endpackage

```

Driver

```

2  package driver_pkg;
3  import shared_pkg::*;
4  import sequenceItem_pkg::*;
5  import uvm_pkg::*;
6  `include "uvm_macros.svh"
7  `define create_obj(type, name) type::type_id::create(name);
8
9  // driver class
10 class ALSU_driver extends uvm_driver #(ALSU_sequenceItem);
11     `uvm_component_utils(ALSU_driver)
12     virtual ALSU_interface v_if;
13     ALSU_sequenceItem stim_seq_item;
14
15     function new(string name = "ALSU_driver", uvm_component parent = null);
16         super.new(name, parent);
17     endfunction //new()
18
19     task run_phase(uvm_phase phase);
20         super.run_phase(phase);
21         forever begin
22             //v_if.inputs_are_driven = 0;
23             stim_seq_item = `create_obj(ALSU_sequenceItem, "stim_seq_item")
24             seq_item_port.get_next_item(stim_seq_item);
25             // assigned inputs to interface
26             v_if.A = stim_seq_item.A;
27             v_if.B = stim_seq_item.B;
28             v_if.cin = stim_seq_item.cin;
29             v_if.serial_in = stim_seq_item.serial_in;
30             v_if.red_op_A = stim_seq_item.red_op_A;
31             v_if.red_op_B = stim_seq_item.red_op_B;
32             v_if.bypass_A = stim_seq_item.bypass_A;
33             v_if.bypass_B = stim_seq_item.bypass_B;
34             v_if.clk = stim_seq_item.clk;
35             v_if.rst = stim_seq_item.rst;
36             v_if.direction = stim_seq_item.direction;
37             // assigned outputs to interface

```

```

37         // assigned opcode to interface
38         v_if.opcode = stim_seq_item.opcode;
39         @(negedge v_if.clk);
40         // assigned outputs to driver
41         stim_seq_item.leds = v_if.leds;
42         stim_seq_item.out = v_if.out;
43         // assigned reference to driver
44         stim_seq_item.leds_ref = v_if.leds_ref;
45         stim_seq_item.out_ref = v_if.out_ref;
46         seq_item_port.item_done();
47         `uvm_info("run_phase_driver", stim_seq_item.convert2string_stim(), UVM_HIGH)
48     end
49 endtask //run_phase
50 endclass //ALSU_driver extends uvm_driver
51
52 endpackage

```

Monitor


```

1  package monitor_pkg;
2  import shared_pkg::*;
3  import sequenceItem_pkg::*;
4  import uvm_pkg::*;
5  `include "uvm_macros.svh"
6  `define create_obj(type, name) type::type_id::create(name);
7  // monitor
8  class ALSU_monitor extends uvm_monitor;
9      `uvm_component_utils(ALSU_monitor)
10     virtual ALSU_interface v_if;
11     ALSU_sequenceItem mon_seq_item;
12     uvm_analysis_port #(ALSU_sequenceItem) mon_port; // monitor is a port
13
14     function new(string name = "ALSU_monitor", uvm_component parent = null);
15         super.new(name, parent);
16     endfunction //new()
17
18     function void build_phase(uvm_phase phase);
19         super.build_phase(phase);
20         mon_port = new("mon_port", this);
21     endfunction
22
23     task run_phase(uvm_phase phase);
24         super.run_phase(phase);
25         forever begin
26             mon_seq_item = `create_obj(ALSU_sequenceItem, "mon_seq_item")
27             //@(posedge v_if.clk);
28             @(negedge v_if.clk);
29             // assigned inputs to monitor
30             mon_seq_item.A = v_if.A;
31             mon_seq_item.B = v_if.B;
32             mon_seq_item.cin = v_if.cin;
33             mon_seq_item.serial_in = v_if.serial_in;
34             mon_seq_item.red_op_A = v_if.red_op_A;
35             mon_seq_item.red_op_B = v_if.red_op_B;
36             mon_seq_item.bypass_A = v_if.bypass_A;
37             mon_seq_item.bypass_B = v_if.bypass_B;

```

```

27 //@(posedge v_if.clk);
28 @(negedge v_if.clk);
29 // assigned inputs to monitor
30     mon_seq_item.A = v_if.A;
31     mon_seq_item.B = v_if.B;
32     mon_seq_item.cin = v_if.cin;
33     mon_seq_item.serial_in = v_if.serial_in;
34     mon_seq_item.red_op_A = v_if.red_op_A;
35     mon_seq_item.red_op_B = v_if.red_op_B;
36     mon_seq_item.bypass_A = v_if.bypass_A;
37     mon_seq_item.bypass_B = v_if.bypass_B;
38     mon_seq_item.clk = v_if.clk;
39     mon_seq_item.rst = v_if.rst;
40     mon_seq_item.direction = v_if.direction;
41 // assigned outputs to monitor
42     mon_seq_item.leds = v_if.leds;
43     mon_seq_item.out = v_if.out;
44 // assigned outputs to monitor
45     mon_seq_item.leds_ref = v_if.leds_ref;
46     mon_seq_item.out_ref = v_if.out_ref;
47 // assigned opcode to monitor
48     mon_seq_item.opcode = opcode_e'(v_if.opcode);
49
50     mon_port.write(mon_seq_item); // that's mean that monitor will send the data
51     `uvm_info("run_phase_monitor", mon_seq_item.convert2string_stim(), UVM_FULL)
52 end
53 endtask //run pha
54 endclass //ALSU_monitor extends uvm_monitor
55 endpackage

```

Agent

```

agent.sv > {} agent_pkg > ALSU_agent
1  `timescale 1ps/1ps
2  package agent_pkg;
3  import shared_pkg::*;
4  import config_pkg::*;
5  import driver_pkg::*;
6  import sequencer_pkg::*;
7  import sequenceItem_pkg::*;
8  import monitor_pkg::*;
9  import uvm_pkg::*;
10 `include "uvm_macros.svh"
11 `define create_obj(type, name) type::type_id::create(name, this);
12 // agent class
13 class ALSU_agent extends uvm_agent;
14     `uvm_component_utils(ALSU_agent)
15     sequencer sqr; // mange data transfer
16     ALSU_driver drv; // inside agent
17     ALSU_monitor mon; // inside agent
18     ALSU_config cfg; // get the data of interface
19     uvm_analysis_port #(ALSU_sequenceItem) agt_port; // agent is a port
20
21     function new(string name = "ALSU_agent", uvm_component parent = null);
22         super.new(name, parent);
23     endfunction //new()
24
25     function void build_phase(uvm_phase phase);
26         super.build_phase(phase);
27
28         if (!uvm_config_db#(ALSU_config)::get(this, "", "CFG", cfg))
29             `uvm_fatal("build_phase", "DRIVER - Unable to get config");
30
31         sqr = `create_obj(sequencer, "sqr")
32         drv = `create_obj(ALSU_driver, "drv")
33         mon = `create_obj(ALSU_monitor, "mon")
34         agt_port = new("agt_port", this);
35     endfunction
36
37     function void connect_phase(uvm_phase phase);
38         drv.v_if = cfg.v_if;
39         mon.v_if = cfg.v_if;
40         drv.seq_item_port.connect(sqr.seq_item_export);
41         mon.mon_port.connect(agt_port);
42     endfunction //connect_phase
43 endclass //ALSU_agent extends uvm_agent
44 endpackage

```

Scoreboard

```

1  package scoreboard_pkg;
2  import agent_pkg::*;
3  import shared_pkg::*;
4  import sequenceItem_pkg::*;
5  import uvm_pkg::*;
6  `include "uvm_macros.svh"
7  `define create_obj(type, name) type::type_id::create(name, this);
8
9  class ALSU_scoreboard extends uvm_scoreboard;
10     `uvm_component_utils(ALSU_scoreboard)
11     uvm_analysis_export #(ALSU_sequenceItem) sb_export; // scoreboard export
12     uvm_tlm_analysis_fifo #(ALSU_sequenceItem) sb_fifo; // scoreboard fifo
13     ALSU_sequenceItem sb_seq_item;
14     // refrence output
15     logic [15:0] leds_ref;
16     logic [5:0] out_ref;
17
18     // error and correct counter
19     int correct_counter = 0;
20     int error_counter = 0;
21
22     function new(string name = "ALSU_scoreboard", uvm_component parent = null);
23     |     super.new(name, parent);
24     endfunction //new()
25
26     function void build_phase(uvm_phase phase);
27     |     super.build_phase(phase);
28     |     sb_export = new("sb_export", this);
29     |     sb_fifo = new("sb_fifo", this);
30     endfunction
31
32     function void connect_phase(uvm_phase phase);
33     |     super.connect_phase(phase);
34     |     sb_export.connect(sb_fifo.analysis_export);
35     endfunction
36

```

```

36
37     task run_phase(uvm_phase phase);
38         super.run_phase(phase);
39
40         forever begin
41             sb_fifo.get(sb_seq_item);
42             //// Checking ////
43             Checking_task(sb_seq_item);
44         end
45     endtask
46
47     task Checking_task(ALSU_sequenceItem chk_item);
48         if (chk_item.out != chk_item.out_ref || chk_item.leds != chk_item.leds_ref) begin
49             error_counter++;
50             `uvm_error("scoreboard",$sformatf("%0s\nout_ref = %0d",chk_item.convert2string(), chk_item.out_ref))
51         end else begin
52             correct_counter++;
53         end
54     endtask //Checking_task
55
56     function void report_phase(uvm_phase phase);
57         super.report_phase(phase);
58         `uvm_info("report_phase", $sformatf("Total correct transaction: %0d", correct_counter), UVM_LOW)
59         `uvm_info("report_phase", $sformatf("Total failed transaction: %0d", error_counter), UVM_LOW)
60     endfunction
61 endclass //ALSU_scoreboard extends uvm_scoreboard
62 endpackage

```

coverage collector

```

1  package coverage_collector_pkg;
2  import agent_pkg::*;
3  import shared_pkg::*;
4  import sequencer_pkg::*;
5  import sequenceItem_pkg::*;
6  import uvm_pkg::*;
7  `include "uvm_macros.svh"
8
9  class ALSU_coverage extends uvm_component;
10     `uvm_component_utils(ALSU_coverage)
11     uvm_analysis_export #(ALSU_sequenceItem) cov_export; // coverage export
12     uvm_tlm_analysis_fifo #(ALSU_sequenceItem) cov_fifo; // coverage fifo
13     ALSU_sequenceItem cov_seq_item;
14
15     //////////////////////////////////////
16     //      begin Coverage Group      //
17     //////////////////////////////////////
18
19     covergroup cvr_gp;
20         // input A bins
21         A_cp: coverpoint cov_seq_item.A {
22             bins A_data_0 = {ZERO};
23             bins A_data_max = {MAXPOS};
24             bins A_data_min = {MAXNEG};
25             bins A_data_default = default;
26             bins A_data_[] = {001, 010, 100};
27         }
28
29         // input B bins
30         B_cp: coverpoint cov_seq_item.B {
31             bins B_data_0 = {ZERO};
32             bins B_data_max = {MAXPOS};
33             bins B_data_min = {MAXNEG};
34             bins B_data_default = default;
35             bins B_data_[] = {001, 010, 100};
36         }
37

```

```

37
38 // cover point for reduction operation red_op
39 op_A_cp: coverpoint cov_seq_item.red_op_A {
40     bins one = {1};
41     bins zero = {0};
42 }
43 op_B_cp: coverpoint cov_seq_item.red_op_B {
44     bins one = {1};
45     bins zero = {0};
46 }
47
48 // Crossing to satisfied the data_walkingones of A and B
49 A_walk: cross A_cp, op_A_cp {
50     option.cross_auto_bin_max = 0;
51     bins A_data_walkingones = binsof(A_cp.A_data_) && binsof(op_A_cp.one);
52 }
53 B_walk: cross B_cp, op_A_cp, op_B_cp {
54     option.cross_auto_bin_max = 0;
55     bins B_data_walkingones = (binsof(B_cp.B_data_)
56                               && binsof(op_A_cp.zero)
57                               && binsof(op_B_cp.one));
58 }
59
60 // cover point for opcode (ALU)
61 ALU_cp: coverpoint cov_seq_item.opcode {
62     bins Bins_shift[] = {SHIFT, ROTATE};
63     bins Bins_arith[] = {ADD, MULT};
64     bins Bins_bitwise[] = {OR, XOR}; //
65     bins Bins_invalid = {INVALID_6, INVALID_7};
66     bins Bins_trans = (0 => 1 => 2 => 3 => 4 => 5);
67 }
68
69 // cover point for c_in
70 cin_cp: coverpoint cov_seq_item.cin;
71

```



```

71
72 // cover point for serial_in
73 serial_cp: coverpoint cov_seq_item.serial_in;
74
75 // cover point for direction
76 direction_cp: coverpoint cov_seq_item.direction;
77
78 // Cross coverage between ALU_cp and A and B
79 ALU_A: cross ALU_cp, A_cp {
80     option.cross_auto_bin_max = 0;
81     bins arith_permutations = binsof(ALU_cp.Bins_arith) && binsof(A_cp) intersect{ZERO, MAXPOS, MAXNEG};
82 }
83 ALU_B: cross ALU_cp, B_cp {
84     option.cross_auto_bin_max = 0;
85     bins arith_permutations = binsof(ALU_cp.Bins_arith) && binsof(B_cp) intersect{ZERO, MAXPOS, MAXNEG};
86 }
87
88 // Cross coverage between ALU_cp and cin_cp
89 ALU_cin: cross ALU_cp, cin_cp {
90     option.cross_auto_bin_max = 0;
91     bins add_cin = binsof(ALU_cp) intersect{ADD};
92 }
93
94 // Cross coverage between ALU_cp and serial_cp
95 ALU_serial: cross ALU_cp, serial_cp {
96     option.cross_auto_bin_max = 0;
97     bins shift_serial = binsof(ALU_cp) intersect{SHIFT};
98 }
99
100 // Cross coverage between ALU_cp and direction_cp
101 ALU_direction: cross ALU_cp, direction_cp {
102     option.cross_auto_bin_max = 0;
103     bins sh_ro_direction = binsof(ALU_cp.Bins_shift);
104 }
105

```

```

106 // Cross coverage ALU = {OR,XOR}, red_op_A = 1, A = data_walk, B = 0
107 A_data_walk_OR_XOR: cross ALU_cp, A_walk, op_A_cp, B_cp {
108     option.cross_auto_bin_max = 0;
109     bins A_walk_OR_XOR = (binsof(ALU_cp.Bins_bitwise)
110         && binsof(A_walk.A_data_walkingones)
111         && binsof(op_A_cp.one)
112         && binsof(B_cp.B_data_0));
113 }
114
115 // Cross coverage ALU = {OR,XOR}, red_op_A = 1, A = data_walk, B = 0
116 B_data_walk_OR_XOR: cross ALU_cp, B_walk, op_B_cp, A_cp {
117     option.cross_auto_bin_max = 0;
118     bins B_walk_OR_XOR = (binsof(ALU_cp.Bins_bitwise)
119         && binsof(B_walk.B_data_walkingones)
120         && binsof(op_B_cp.one)
121         && binsof(A_cp.A_data_0));
122 }
123
124 // Cross coverage Invalid case 2 red_op
125 Invalid_red_op: cross ALU_cp, op_A_cp, op_B_cp {
126     option.cross_auto_bin_max = 0;
127     bins Invalid_reduction = (binsof(ALU_cp.Bins_bitwise)
128         && (binsof(op_A_cp.one) || binsof(op_B_cp.one)));
129 }
130
131 // Invalid case with reduction operation
132 reduction_invalid: cross ALU_cp, op_A_cp, op_B_cp {
133     option.cross_auto_bin_max = 0;
134     bins invalid_red_op = (binsof(ALU_cp) intersect{!OR, !XOR} && (binsof(op_A_cp.one) || binsof(op_B_cp.one)));
135 }
136

```

```

138         rst_cp: coverpoint cov_seq_item.rst;
139
140         // Cross coverage red_op_A and red_op_B
141         red_op_High_cross: cross op_A_cp, op_B_cp;
142     endgroup
143
144     //////////////////////////////////////
145     //      finish Coverage Group      //
146     //////////////////////////////////////
147
148     function new(string name = "ALSU_coverage", uvm_component parent = null);
149         super.new(name, parent);
150         cvr_gp = new();
151     endfunction //new()
152
153     function void build_phase(uvm_phase phase);
154         super.build_phase(phase);
155         cov_export = new("cov_export", this);
156         cov_fifo = new("cov_fifo", this);
157     endfunction
158
159     function void connect_phase(uvm_phase phase);
160         super.connect_phase(phase);
161         cov_export.connect(cov_fifo.analysis_export);
162     endfunction
163
164     task run_phase(uvm_phase phase);
165         super.run_phase(phase);
166         forever begin
167             cov_fifo.get(cov_seq_item);
168             cvr_gp.sample();
169         end
170     endtask
171 endclass
172 endpackage

```

Environment

```

1  package env_pkg;
2  import shared_pkg::*;
3  import scoreboard_pkg::*;
4  import coverage_collector_pkg::*;
5  import agent_pkg::*;
6  import uvm_pkg::*;
7  `include "uvm_macros.svh"
8  `define create_obj(type, name) type::type_id::create(name, this);
9
10 class ALSU_env extends uvm_env;
11     `uvm_component_utils(ALSU_env)
12
13     ALSU_scoreboard sb;
14     ALSU_coverage cov;
15     ALSU_agent agt;
16
17     function new(string name = "ALSU_env", uvm_component parent = null);
18         super.new(name, parent);
19     endfunction
20
21     function void build_phase(uvm_phase phase);
22         super.build_phase(phase);
23
24
25         agt = `create_obj(ALSU_agent, "agt")
26         sb = `create_obj(ALSU_scoreboard, "sb")
27         cov = `create_obj(ALSU_coverage, "cov")
28
29     endfunction
30
31
32     function void connect_phase(uvm_phase phase);
33         agt.agt_port.connect(sb.sb_export);
34         agt.agt_port.connect(cov.cov_export);
35     endfunction
36 endclass
37 endpackage

```

Test

test.sv > {} test_pkg > ALSU_test

```
1  package test_pkg;
2  import shared_pkg::*;
3  import env_pkg::*;
4  import config_pkg::*;
5  import main_sequence_pkg::*;
6  import rst_sequence_pkg::*;
7  import sequenceItem_pkg::*;
8  import sequenceItem_valid_pkg::*;
9
10 import uvm_pkg::*;
11 `include "uvm_macros.svh"
12 `define create_obj(type, name) type::type_id::create(name, this);
13
14 class ALSU_test extends uvm_test;
15     `uvm_component_utils(ALSU_test)
16     ALSU_env env;
17     ALSU_config cfg;
18
19     ALSU_reset_sequence rst_seq;
20     ALSU_main_sequence main_seq;
21
22     function new(string name = "ALSU_test", uvm_component parent = null);
23         super.new(name, parent);
24     endfunction
25
26     function void build_phase(uvm_phase phase);
27         uvm_factory factory = uvm_factory::get();
28         super.build_phase(phase);
29
30         factory.print();
31         env = `create_obj(ALSU_env, "env");
32         cfg = `create_obj(ALSU_config, "cfg");
33         rst_seq = `create_obj(ALSU_reset_sequence, "rst_seq");
34         main_seq = `create_obj(ALSU_main_sequence, "main_seq");
35
36         if (!uvm_config_db#(virtual ALSU_interface)::get(this, "", "INTERFACE", cfg.v_if))
37             `uvm_fatal("build_phase", "TEST - Unable to get config");
```

```

29
30     factory.print();
31     env = `create_obj(ALSU_env, "env");
32     cfg = `create_obj(ALSU_config, "cfg");
33     rst_seq = `create_obj(ALSU_reset_sequence, "rst_seq")
34     main_seq = `create_obj(ALSU_main_sequence, "main_seq")
35
36     if (!uvm_config_db#(virtual ALSU_interface)::get(this, "", "INTERFACE", cfg.v_if))
37         `uvm_fatal("build_phase", "TEST - Unable to get config");
38
39     uvm_config_db#(ALSU_config)::set(this, "*", "CFG", cfg);
40 endfunction
41
42 task run_phase(uvm_phase phase);
43     super.run_phase(phase);
44
45     phase.raise_objection(this);
46
47     #1; `uvm_info("run_phase", "Inside the slaby test DEBUG", UVM_DEBUG)
48
49     ////////////////
50     // stimulus //
51     ////////////////
52     // rst seq
53     `uvm_info("run_phase", "ALSU reset seq start", UVM_MEDIUM)
54     rst_seq.start(env.agt.sqr);
55     `uvm_info("run_phase", "ALSU reset seq finish", UVM_MEDIUM)
56     // Main seq
57     `uvm_info("run_phase", "ALSU main seq start", UVM_MEDIUM)
58     main_seq.start(env.agt.sqr);
59     `uvm_info("run_phase", "ALSU reset seq finish", UVM_MEDIUM)
60
61     phase.drop_objection(this);
62 endtask
63 endclass
64 endpackage

```

Assertion

```

1  import shared_pkg::*;
2  `define reset_disable iff(rst)
3  module ALSU_sva(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
4  input bit clk;
5  input logic rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
6  input [2:0] opcode;
7  input logic signed [2:0] A, B;
8  input logic [15:0] leds;
9  input logic [5:0] out;
10 ///////////////////////////////////////////////////////////////////
11 // this variable to make writing assertion more easier //
12 ///////////////////////////////////////////////////////////////////
13 bit isBypass, InvalidOP, bitwise;
14 assign isBypass = bypass_A | bypass_B;
15 assign InvalidOP = opcode==6 | opcode==7;
16 assign bitwise = opcode==0 | opcode==1;
17 assign isRed = red_op_B | red_op_A;
18
19 ///////////////////////////////////////////////////////////////////
20 // assertion of bypass operation //
21 ///////////////////////////////////////////////////////////////////
22 bypass_A_assert: assert property (@(posedge clk) `reset bypass_A |-> ##2 out==$past(A,2));
23 bypass_B_assert: assert property (@(posedge clk) `reset (bypass_B && !bypass_A) |-> ##2 out==$past(B,2));
24
25 ///////////////////////////////////////////////////////////////////
26 // assertion of reduction operation //
27 ///////////////////////////////////////////////////////////////////
28 sequence redValid(red, op,red2);
29 | | (red && !isBypass && opcode==op && !red2);
30 endsequence
31
32 redA_OR_assert: assert property (@(posedge clk) `reset redValid(red_op_A,0,0) |-> ##2 out==|($past(A,2)));
33 redB_OR_assert: assert property (@(posedge clk) `reset redValid(red_op_B,0,red_op_A) |-> ##2 out==|($past(B,2)));
34
35 redA_XR_assert: assert property (@(posedge clk) `reset redValid(red_op_A,1,0) |-> ##2 out==^($past(A,2)));
36 redB_XR_assert: assert property (@(posedge clk) `reset redValid(red_op_B,1,red_op_A) |-> ##2 out==^($past(B,2)));
37

```

Acti
Go to

```

37
38 ///////////////////////////////////////////////////////////////////
39 // assertion of invalid cases //
40 ///////////////////////////////////////////////////////////////////
41 sequence Invalid_seq;
42 | (isRed && !isBypass && !bitwise) || (InvalidOP && !isBypass));
43 endsequence
44
45 Invalid_assert: assert property (@(posedge clk) `reset Invalid_seq |-> ##[0:2](out==0 && leds=='hffff'));
46
47 ///////////////////////////////////////////////////////////////////
48 // assertion when opcode always valid //
49 ///////////////////////////////////////////////////////////////////
50
51 always_comb begin : assert_alwaysValid
52     if (alwaysValid) begin
53         alwValid_opcode_assert: assert final (!InvalidOP);
54         alwZero_bypass_A_assert: assert final (!bypass_A);
55         alwZero_bypass_B_assert: assert final (!bypass_B);
56         alwZero_red_op_A_assert: assert final (!red_op_A);
57         alwZero_red_op_B_assert: assert final (!red_op_B);
58     end
59 end
60
61 ///////////////////////////////////////////////////////////////////
62 // reset assertion //
63 ///////////////////////////////////////////////////////////////////
64 always_comb begin : reset_assertion
65     if (rst) begin
66         rst_out_assert: assert final (out==0);
67         rst_leds_assert: assert final (leds==0);
68     end
69 end
70
71 ///////////////////////////////////////////////////////////////////
72 // assertion for opcode Valid cases without bypass or reduction //
73 ///////////////////////////////////////////////////////////////////
74 sequence sh_ro(op, dir);
75 | (opcode==op && dir && !isRed && !isBypass);
76 endsequence
77 sequence op_assert(op);
78 | (!isBypass && !isRed && opcode==op);
79 endsequence
80
81 OR_assert: assert property (@(posedge clk) `reset op_assert(0) |-> ##2 out== $past(A,2) | $past(B,2) );
82 XOR_assert: assert property (@(posedge clk) `reset op_assert(1) |-> ##2 out==($past(A,2)^$past(B,2)) );
83 add_assert: assert property (@(posedge clk) `reset op_assert(2) |-> ##2 out==($past(A,2)+$past(B,2)+$past(cin,2)) );
84 mult_assert: assert property (@(posedge clk) `reset op_assert(3) |-> ##2 out==($past(A,2)*$past(B,2)) );
85
86 shiftL_assert: assert property (@(posedge clk) `reset sh_ro(4, direction) |-> ##2 out=={$past(out[4:0]), $past(serial_in,2)} );
87 shiftR_assert: assert property (@(posedge clk) `reset sh_ro(4, !direction) |-> ##2 out=={$past(serial_in,2), $past(out[5:1])} );
88
89 rotateL_assert: assert property (@(posedge clk) `reset sh_ro(5, direction) |-> ##2 out=={$past(out[4:0]), $past(out[5])} );
90 rotateR_assert: assert property (@(posedge clk) `reset sh_ro(5, !direction) |-> ##2 out=={$past(out[0]), $past(out[5:1])} );
91
92 ///////////////////////////////////////////////////////////////////
93 // cover //
94 ///////////////////////////////////////////////////////////////////
95
96 ///////////////////////////////////////////////////////////////////
97 // cover of bypass operation //
98 ///////////////////////////////////////////////////////////////////
99 bypass_A_cover: cover property (@(posedge clk) `reset bypass_A |-> ##2 out==$past(A,2));
100 bypass_B_cover: cover property (@(posedge clk) `reset (bypass_B && !bypass_A) |-> ##2 out==$past(B,2));
101

```

```

102 ///////////////////////////////////////////////////
103 // cover of reduction operation //
104 ///////////////////////////////////////////////////
105
106 redA_OR_cover: cover property (@(posedge clk) `reset redValid(red_op_A,0,0) |-> ##2 out==|($past(A,2)));
107 redB_OR_cover: cover property (@(posedge clk) `reset redValid(red_op_B,0,red_op_A) |-> ##2 out==|($past(B,2)));
108
109 redA_XR_cover: cover property (@(posedge clk) `reset redValid(red_op_A,1,0) |-> ##2 out==^($past(A,2)));
110 redB_XR_cover: cover property (@(posedge clk) `reset redValid(red_op_B,1,red_op_A) |-> ##2 out==^($past(B,2)));
111
112 ///////////////////////////////////////////////////
113 // cover of invalid cases //
114 ///////////////////////////////////////////////////
115 Invalid_cover: cover property (@(posedge clk) `reset Invalid_seq |-> ##[0:2](out==0 && leds=='hfff));
116
117 ///////////////////////////////////////////////////
118 // cover when opcode always valid //
119 ///////////////////////////////////////////////////
120
121 always_comb begin : cover_alwaysValid
122     if (alwaysValid) begin
123         alwValid_opcode_cover: cover final (!InvalidOP);
124         alwZero_bypass_A_cover: cover final (!bypass_A);
125         alwZero_bypass_B_cover: cover final (!bypass_B);
126         alwZero_red_op_A_cover: cover final (!red_op_A);
127         alwZero_red_op_B_cover: cover final (!red_op_B);
128     end
129 end
130
131 ///////////////////////////////////////////////////
132 // reset cover //
133 ///////////////////////////////////////////////////
134 always_comb begin : reset_cover
135     if (rst) begin
136         rst_out_cover: cover final (out==0);
137         rst_leds_cover: cover final (leds==0);
138     end
139 end
140
141 ///////////////////////////////////////////////////
142 // cover for opcode Valid cases without bypass or reduction //
143 ///////////////////////////////////////////////////
144
145 OR_cover: cover property (@(posedge clk) `reset op_assert(0) |-> ##2 out== $past(A,2) | $past(B,2) );
146 XOR_cover: cover property (@(posedge clk) `reset op_assert(1) |-> ##2 out==($past(A,2) ^ $past(B,2) );
147 add_cover: cover property (@(posedge clk) `reset op_assert(2) |-> ##2 out==($past(A,2) + $past(B,2) + $past(cin,2) );
148 mult_cover: cover property (@(posedge clk) `reset op_assert(3) |-> ##2 out==($past(A,2) * $past(B,2) );
149
150 shiftL_cover: cover property (@(posedge clk) `reset sh_ro(4, direction) |-> ##2 out=={$past(out[4:0]), $past(serial_in,2)} );
151 shiftR_cover: cover property (@(posedge clk) `reset sh_ro(4, !direction) |-> ##2 out=={$past(serial_in,2), $past(out[5:1]) } );
152
153 rotateL_cover: cover property (@(posedge clk) `reset sh_ro(5, direction) |-> ##2 out=={$past(out[4:0]), $past(out[5]) } );
154 rotateR_cover: cover property (@(posedge clk) `reset sh_ro(5, !direction) |-> ##2 out=={$past(out[0]), $past(out[5:1]) } );
155 endmodule

```

Top


```

1  timescale 1ps/1ps
2  import uvm_pkg::*;
3  `include "uvm_macros.svh"
4  import test_pkg::*;
5  module top ();
6      bit clk;
7
8      initial begin
9          forever #1 clk = ~clk;
10     end
11
12     ALSU_interface intf (clk);
13     ALSU DUT (
14         intf.A, intf.B, intf.cin, intf.serial_in, intf.red_op_A,
15         intf.red_op_B, intf.opcode, intf.bypass_A, intf.bypass_B,
16         intf.clk, intf.rst, intf.direction, intf.leds, intf.out
17     );
18
19     ALSU_ref GLD (
20         intf.A, intf.B, intf.cin, intf.serial_in, intf.red_op_A,
21         intf.red_op_B, intf.opcode, intf.bypass_A, intf.bypass_B,
22         intf.clk, intf.rst, intf.direction, intf.leds_ref, intf.out_ref
23     );
24
25     bind ALSU ALSU_sva ALSU_sva_inst(
26         A, B, cin, serial_in, red_op_A, red_op_B,
27         opcode, bypass_A, bypass_B, clk, rst, direction, leds, out
28     );
29
30     initial begin
31         uvm_config_db#(virtual ALSU_interface)::set(null, "uvm_test_top", "INTERFACE", intf);
32         run_test("ALSU_test");
33     end
34 endmodule

```

Do file

```

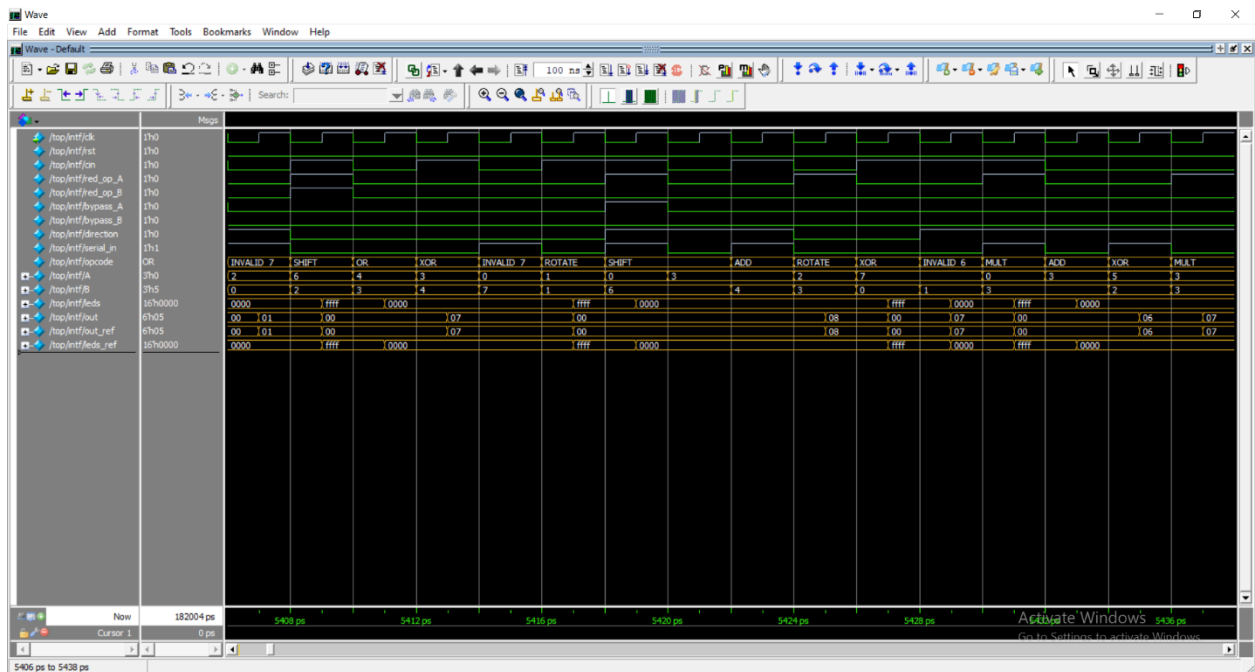
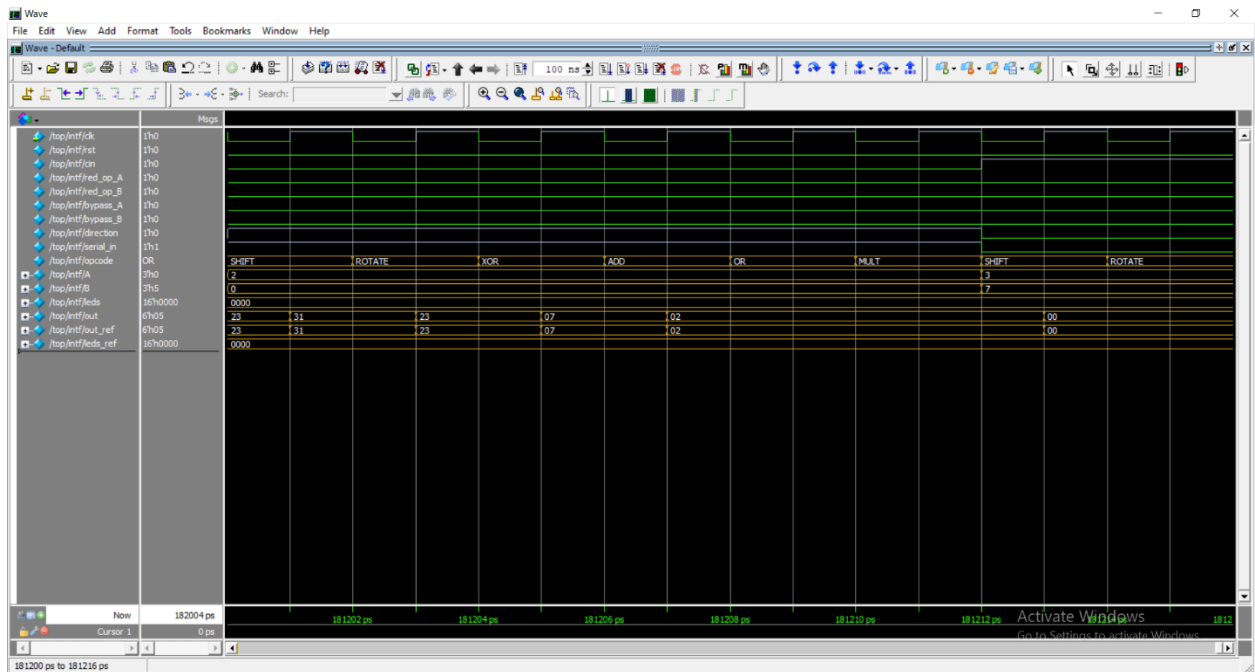
1  vlib work
2  vlog -f src_files.list
3  vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all
4  add wave /top/intf/*
5  run -all

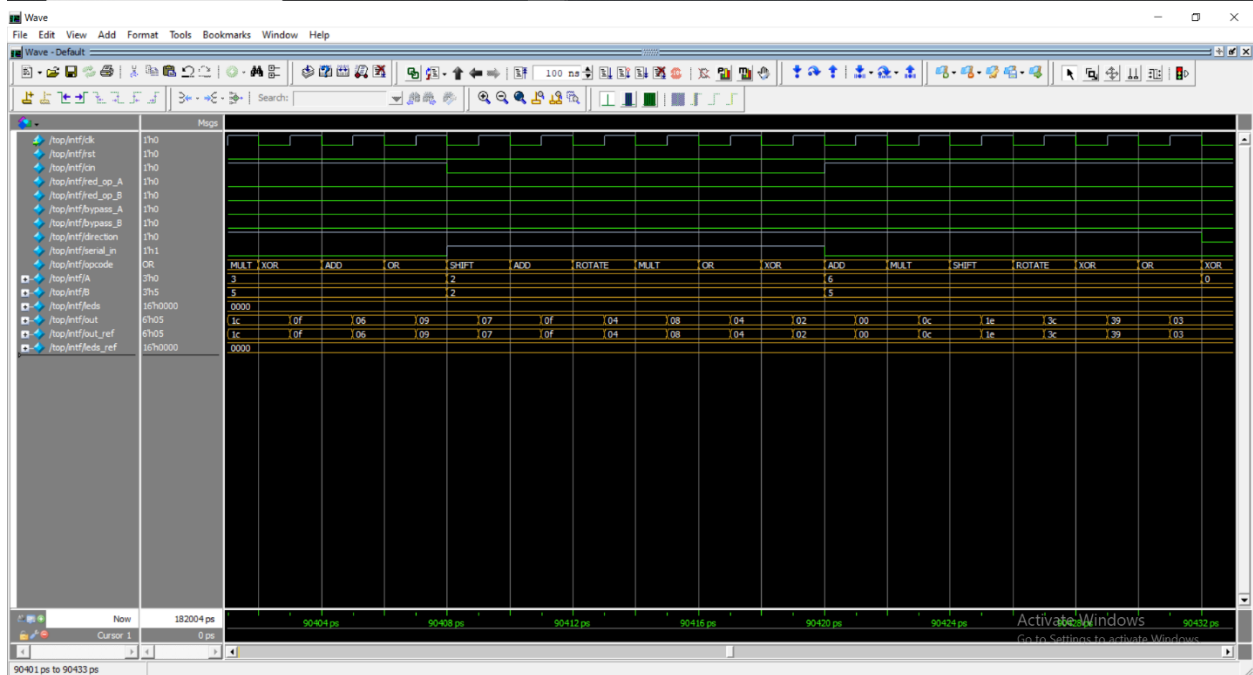
```

Src_List

```
1  ALSU.sv
2  ALSU_ref.sv
3  interface.sv
4  shared_pkg.sv
5  configuration.sv
6  sequenceItem.sv
7  sequenceItem_Valid.sv
8  ALSU_reset_sequence.sv
9  ALSU_main_sequence.sv
10 sequencer.sv
11 driver.sv
12 monitor.sv
13 agent.sv
14 scoreboard.sv
15 coverage_collector.sv
16 env.sv
17 test.sv
18 assertion.sv
19 top.sv
20
21
```

wave form





cover groups

Covergroups

File Bookmarks Window Help

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/coverage_collector_pkg/ALSU_coverage		100.00%							
TYPE cvr_gp		100.00%	100	100.00...		✓	auto(0)		
CVP cvr_gp::A_cp		100.00%	100	100.00...		✓			
CVP cvr_gp::B_cp		100.00%	100	100.00...		✓			
CVP cvr_gp::op_A_cp		100.00%	100	100.00...		✓			
CVP cvr_gp::op_B_cp		100.00%	100	100.00...		✓			
CVP cvr_gp::ALU_cp		100.00%	100	100.00...		✓			
CVP cvr_gp::dn_cp		100.00%	100	100.00...		✓			
CVP cvr_gp::serial_cp		100.00%	100	100.00...		✓			
CVP cvr_gp::direction_cp		100.00%	100	100.00...		✓			
CVP cvr_gp::rst_cp		100.00%	100	100.00...		✓			
CROSS cvr_gp::A_walk		100.00%	100	100.00...		✓			
CROSS cvr_gp::B_walk		100.00%	100	100.00...		✓			
CROSS cvr_gp::ALU_A		100.00%	100	100.00...		✓			
CROSS cvr_gp::ALU_B		100.00%	100	100.00...		✓			
CROSS cvr_gp::ALU_dn		100.00%	100	100.00...		✓			
CROSS cvr_gp::ALU_serial		100.00%	100	100.00...		✓			
CROSS cvr_gp::ALU_direction		100.00%	100	100.00...		✓			
CROSS cvr_gp::A_data_walk_OR_XOR		100.00%	100	100.00...		✓			
CROSS cvr_gp::B_data_walk_OR_XOR		100.00%	100	100.00...		✓			
CROSS cvr_gp::Invalid_red_op		100.00%	100	100.00...		✓			
CROSS cvr_gp::reduction_invalid		100.00%	100	100.00...		✓			
CROSS cvr_gp::red_op_High_cross		100.00%	100	100.00...		✓			
INST /coverage_collector_pkg::ALSU_coverage::cvr_gp		100.00%	100	100.00...		✓			0

Covergroups

File Bookmarks Window Help

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
INST /coverage_collector_pkg::ALSU_coverage::cvr_gp		100.00%	100	100.00...		✓			0
CVP A_cp		100.00%	100	100.00...		✓			
bin A_data_0		13221	1	100.00...		✓			
bin A_data_max		11761	1	100.00...		✓			
bin A_data_min		11973	1	100.00...		✓			
bin A_data_1		10843	1	100.00...		✓			
default bin A_data_default		43204	-	-		✓			
CVP B_cp		100.00%	100	100.00...		✓			
bin B_data_0		12820	1	100.00...		✓			
bin B_data_max		12035	1	100.00...		✓			
bin B_data_min		12166	1	100.00...		✓			
bin B_data_1		10606	1	100.00...		✓			
default bin B_data_default		43375	-	-		✓			
CVP op_A_cp		100.00%	100	100.00...		✓			
bin one		6337	1	100.00...		✓			
bin zero		84665	1	100.00...		✓			
CVP op_B_cp		100.00%	100	100.00...		✓			
bin one		6444	1	100.00...		✓			
bin zero		84558	1	100.00...		✓			
CVP ALU_cp		100.00%	100	100.00...		✓			
bin Bins_shift[SHIFT]		14628	1	100.00...		✓			
bin Bins_shift[ROTATE]		14582	1	100.00...		✓			
bin Bins_arith[ADD]		14668	1	100.00...		✓			
bin Bins_arith[MULT]		14646	1	100.00...		✓			
bin Bins_bitwise[OR]		14577	1	100.00...		✓			
bin Bins_bitwise[XOR]		14598	1	100.00...		✓			
bin Bins_invalid		3303	1	100.00...		✓			
bin Bins_trans		19	1	100.00...		✓			
CVP dn_cp		100.00%	100	100.00...		✓			
bin auto[0]		45081	1	100.00...		✓			
bin auto[1]		45921	1	100.00...		✓			
CVP serial_cp		100.00%	100	100.00...		✓			
bin auto[0]		45532	1	100.00...		✓			
bin auto[1]		45470	1	100.00...		✓			
CVP direction_cp		100.00%	100	100.00...		✓			
bin auto[0]		45407	1	100.00...		✓			
bin auto[1]		45595	1	100.00...		✓			
CVP rst_cp		100.00%	100	100.00...		✓			
bin auto[0]		89026	1	100.00...		✓			
bin auto[1]		1976	1	100.00...		✓			

Covergroups										
File Bookmarks Window Help										
Covergroups										
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment	
bin Bins_bitwise[XOR]		14598	1	100.00...						
bin Bins_invalid		3303	1	100.00...						
bin Bins_trans		19	1	100.00...						
CVP cin_cp		100.00%	100	100.00...						
bin auto[0]		45081	1	100.00...						
bin auto[1]		45921	1	100.00...						
CVP serial_cp		100.00%	100	100.00...						
bin auto[0]		45532	1	100.00...						
bin auto[1]		45470	1	100.00...						
CVP direction_cp		100.00%	100	100.00...						
bin auto[0]		45407	1	100.00...						
bin auto[1]		45595	1	100.00...						
CVP rst_cp		100.00%	100	100.00...						
bin auto[0]		89026	1	100.00...						
bin auto[1]		1976	1	100.00...						
CROSS A_walk		100.00%	100	100.00...						
bin A_data_walkingones		999	1	100.00...						
CROSS B_walk		100.00%	100	100.00...						
bin B_data_walkingones		821	1	100.00...						
CROSS ALU_A		100.00%	100	100.00...						
bin arith_permutations		12870	1	100.00...						
CROSS ALU_B		100.00%	100	100.00...						
bin arith_permutations		12858	1	100.00...						
CROSS ALU_cin		100.00%	100	100.00...						
bin add_cin		14668	1	100.00...						
CROSS ALU_serial		100.00%	100	100.00...						
bin shift_serial		14628	1	100.00...						
CROSS ALU_direction		100.00%	100	100.00...						
bin sh_ro_direction		29210	1	100.00...						
CROSS A_data_walk_OR_XOR		100.00%	100	100.00...						
bin A_walk_OR_XOR		1543	1	100.00...						
CROSS B_data_walk_OR_XOR		100.00%	100	100.00...						
bin B_walk_OR_XOR		1579	1	100.00...						
CROSS Invalid_red_op		100.00%	100	100.00...						
bin Invalid_reduction		3232	1	100.00...						
CROSS reduction_invalid		100.00%	100	100.00...						
bin invalid_red_op		3232	1	100.00...						
CROSS red_op_High_cross		100.00%	100	100.00...						
bin <zero,zero>		79703	1	100.00...						
bin <one,zero>		4855	1	100.00...						
bin <zero,one>		4962	1	100.00...						
bin <one,one>		1482	1	100.00...						

Assertions

main_sequence_pkg::ALSU_main_sequence::body/#ubk#4...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	
main_sequence_pkg::ALSU_main_sequence::body/#ubk#4...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	
main_sequence_pkg::ALSU_main_sequence::body/#ubk#4...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	
top/DUT/ALSU_sva_inst/bypass_A_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) ((bypass_A) <-> 1) #2 [out == \$pass...	
top/DUT/ALSU_sva_inst/bypass_B_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((bypass_B) <-> bypass_A)) <-> 1 #...	
top/DUT/ALSU_sva_inst/redA_OR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & apoc...	
top/DUT/ALSU_sva_inst/redA_OR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & apoc...	
top/DUT/ALSU_sva_inst/redA_OR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & apoc...	
top/DUT/ALSU_sva_inst/redB_OR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_B) <-> bypass)) & apoc...	
top/DUT/ALSU_sva_inst/redB_OR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_B) <-> bypass)) & apoc...	
top/DUT/ALSU_sva_inst/invalid_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~bypass...	
top/DUT/ALSU_sva_inst/OR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~red) & apoc...	
top/DUT/ALSU_sva_inst/OR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~red) & apoc...	
top/DUT/ALSU_sva_inst/add_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~red) & apoc...	
top/DUT/ALSU_sva_inst/mult_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~red) & apoc...	
top/DUT/ALSU_sva_inst/shift_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~red) & apoc...	
top/DUT/ALSU_sva_inst/shiftR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~red) & apoc...	
top/DUT/ALSU_sva_inst/rotateL_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~red) & apoc...	
top/DUT/ALSU_sva_inst/rotateR_assert	Concurrent	SVA	on	0	1	-	0B	0B	0 ps	0 off	assert (@(posedge ck) disable ff (rst) (((red_op_A) <-> bypass)) & ~red) & apoc...	
top/DUT/ALSU_sva_inst/assert_alwaysValid/alwaysValid_opcod...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (~invalidOP)	
top/DUT/ALSU_sva_inst/assert_alwaysValid/alwaysValid_opcod...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (~invalidOP)	
top/DUT/ALSU_sva_inst/assert_alwaysValid/alwaysValid/alwaysValidZero_bypas...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (~bypass_A)	
top/DUT/ALSU_sva_inst/assert_alwaysValid/alwaysValid/alwaysValidZero_bypas...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (~bypass_B)	
top/DUT/ALSU_sva_inst/assert_alwaysValid/alwaysValid/alwaysValidZero_red_o...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (~red_op_A)	
top/DUT/ALSU_sva_inst/assert_alwaysValid/alwaysValid/alwaysValidZero_red_o...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (~red_op_B)	
top/DUT/ALSU_sva_inst/reset_assert/rst_out_assert	Immediate	SVA	on	0	1	-	-	-	-	off	assert (out == 0)	
top/DUT/ALSU_sva_inst/reset_assert/rst_leds_assert	Immediate	SVA	on	0	1	-	-	-	-	off	assert (leds == 0)	